

PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO

Relatório Técnico — Atividade Avaliativa Final

Sistema de Notas e Médias de Alunos (Problema 1)

Disciplina:	Paradigmas de Linguagens de Programação
Professor:	Rogério Mandelli
Turma:	4169EGSN1A2_P1 — Campus Tijuca
Grupo:	1
Membros (Nome e RA):	1. André Nunes Sirqueira - 1260129942 2. Tarcisio Silva Barboza - 1260126680 3. Gabriel de Oliveira Fórnes - 1260129775 4. Gabriel Barboteu Malfacini - 1260129888
Data de entrega:	01/07/2026

1. Introdução Teórica

Esta seção apresenta os principais conceitos dos paradigmas de programação, das linguagens utilizadas e dos fundamentos que serviram de base para a comparação realizada neste relatório.

1.1 Paradigmas de Programação

Um paradigma de programação é um conjunto de conceitos que define a forma de organizar e desenvolver um programa. Cada paradigma possui uma maneira diferente de estruturar os dados, controlar a execução e resolver problemas. Neste trabalho, foram utilizados três paradigmas:

- **Estruturado:** organiza o código em funções, utilizando estruturas de sequência, decisão e repetição. É um paradigma simples, com foco na modularização e na separação das responsabilidades.
- **Orientado a Objetos (POO):** organiza o sistema em objetos que possuem atributos e métodos. Utiliza conceitos como encapsulamento, herança e polimorfismo para representar melhor as entidades do problema e facilitar a manutenção do código.
- **Funcional:** baseia-se no uso de funções puras, imutabilidade e funções de alta ordem, como map, filter, reduce e lambda. Seu objetivo é tornar o código mais previsível, reutilizável e com menos efeitos colaterais.

1.2 Linguagens Utilizadas

Para a implementação prática do problema escolhido, foram utilizadas duas linguagens, conforme orientação do enunciado:

- Python: linguagem multiparadigma, interpretada e de tipagem dinâmica, utilizada nas implementações Estruturada e Declarativa/Funcional, por oferecer suporte nativo a funções de alta ordem (map, filter, reduce) e expressões lambda.
- Java: linguagem orientada a objetos, compilada e de tipagem estática, utilizada na implementação Orientada a Objetos, por oferecer suporte robusto a classes, herança, interfaces, polimorfismo e tratamento de exceções checadas.

1.3 Conceitos Fundamentais da Disciplina

- Tipos de dados e tipagem;
- Escopo de variáveis;
- Modularização;
- Tratamento de exceções;
- Imutabilidade e funções puras;
- Persistência de dados;

2. Descrição do Problema

2.1 Problema Escolhido

O grupo escolheu o Problema 1 — Sistema de Notas e Médias de Alunos.

2.2 Contexto

É um sistema acadêmico simples que registra as notas dos alunos, calcula a média individual e da turma, identifica a maior e a menor nota e informa se cada aluno foi aprovado ou reprovado. Esse tipo de sistema representa uma versão simplificada dos sistemas de gestão escolar usados para acompanhar o desempenho dos estudantes.

2.3 Requisitos

O sistema deve atender aos seguintes requisitos funcionais:

- Ler nome e notas de n alunos;
- Calcular a média individual de cada aluno;
- Determinar a situação do aluno (Aprovado, se média $\geq 6,0$; Reprovado, caso contrário);
- Exibir a listagem de alunos de forma ordenada;
- Calcular a média geral da turma;
- Identificar a maior nota registrada;
- Identificar a menor nota registrada.

2.4 Objetivo da Solução

O objetivo é resolver o mesmo problema utilizando três abordagens paradigmáticas diferentes: Estruturada, Orientada a Objetos e Declarativa/Funcional, de modo a evidenciar, na prática, como cada paradigma influencia a organização do código, a legibilidade, a reutilização e a forma de manipular dados e comportamento, consolidando os conceitos teóricos estudados ao longo da disciplina.

3. Implementação Comentada

3.1 Implementação Estruturada (Python)

Arquivo: Trabalho_de_paradigmas_-_Sistemas_de_notas_(Estruturado).py

A versão estruturada foi implementada em Python, utilizando funções independentes que operam sobre estruturas de dados simples (listas de dicionários), sem o uso de classes. O fluxo do programa segue um modelo procedural clássico: entrada de dados → processamento → ordenação → saída de dados.

Principais funções e suas responsabilidades:

- `ler_alunos(n)`: realiza a entrada de dados, lendo nome e três notas de cada aluno, com validação de intervalo (0 a 10) e tratamento de erros de digitação por meio de laços `while` e blocos `try/except`;
- `calcular_media(notas)`: função utilitária que calcula a média aritmética de uma lista de notas;
- `determinar_situacao(media)`: aplica a regra de negócio que define Aprovado ($\text{média} \geq 6$) ou Reprovado;
- `processar_alunos(alunos)`: percorre a lista de alunos, enriquecendo cada dicionário com as chaves média e situação;
- `ordenar_alunos(alunos)`: retorna a lista ordenada de forma decrescente por média, utilizando `sorted()` com função-chave (`key`);
- `exibir_listagem(alunos)` e `calcular_estatisticas(alunos)`: responsáveis pela apresentação dos resultados e pelo cálculo de média geral, maior e menor nota da turma;
- `main()`: função principal que orquestra a chamada das demais funções em sequência, demonstrando a separação lógica das tarefas (modularização).

Essa estrutura evidencia os pilares do paradigma estruturado: decomposição do problema em sub-rotinas (funções), uso de estruturas de controle (`for`, `while`, `if`) e ausência de encapsulamento de estado — os dados (dicionários de alunos) trafegam explicitamente como parâmetros entre as funções.

3.2 Implementação Orientada a Objetos (Java)

Arquivos: Pessoa.java, Aluno.java, Turma.java, Relatorio.java, RelatorioTurma.java, NotaInvalidaException.java, Main.java

A versão orientada a objetos foi implementada em Java, modelando o domínio do problema por meio de classes que representam as entidades Pessoa, Aluno, Turma e Relatório, além de uma exceção customizada para validação de notas. A organização do código segue os princípios de abstração, encapsulamento, herança e polimorfismo.

Estrutura geral das classes:

- `Pessoa` (classe abstrata): define o atributo `nome` (encapsulado por getters/setters) e declara o método abstrato `exibirInformacoes()`, que deve ser implementado pelas subclasses. Representa a abstração de uma pessoa genérica;
- `Aluno` (extends `Pessoa`, implements `Comparable<Aluno>`): herda de `Pessoa` e adiciona a lista de notas, os métodos de cálculo (`calcularMedia`, `getMaiorNota`, `getMenorNota`) e a lógica de situação (`getSituacao`). Implementa `Comparable` para permitir ordenação por nome via `Collections.sort()`, e sobrescreve `exibirInformacoes()` (polimorfismo). O método `adicionarNota()` valida a nota e lança `NotaInvalidaException` em caso de valor inválido;

- `NotaInvalidaException` (extends `Exception`): exceção checada customizada, lançada quando uma nota informada está fora do intervalo de 0,0 a 10,0, demonstrando o uso de tratamento de exceções de domínio;
- `Turma`: encapsula a coleção de objetos `Aluno` e concentra a lógica de estatísticas da turma (média geral, maior nota, menor nota, total de aprovados/reprovados), exemplificando o princípio de composição;
- `Relatorio` (classe abstrata) e `RelatorioTurma` (extends `Relatorio`): aplicam o padrão de projeto `Template Method` — a classe `Relatorio` define o método final `gerar()`, que chama em sequência os métodos abstratos `exibirCabecalho()`, `exibirCorpo()` e `exibirRodape()`, implementados de forma concreta em `RelatorioTurma`. Essa estrutura demonstra herança e polimorfismo em sua forma mais clara dentro do projeto;
- `Main`: ponto de entrada do sistema, responsável pela leitura interativa dos dados via `Scanner`, com tratamento de exceções (`try/catch/finally`) tanto para erros de entrada (`InputMismatchException`) quanto para a exceção de domínio (`NotaInvalidaException`), permitindo nova tentativa de digitação sem interromper a execução.

Essa organização evidencia os quatro pilares da Orientação a Objetos: abstração (classes abstratas `Pessoa` e `Relatorio`), encapsulamento (atributos privados com acesso controlado), herança (`Aluno` herda de `Pessoa`; `RelatorioTurma` herda de `Relatorio`) e polimorfismo (sobrescrita de métodos abstratos e uso de `Relatorio relatorio = new RelatorioTurma(turma)`, em que um objeto é referenciado por seu tipo mais genérico).

3.3 Implementação Declarativa/Funcional (Python)

Arquivo: Trabalho_de_paradigmas_-_Sistemas_de_notas_(Declarativo).py

A versão declarativa/funcional foi implementada em Python utilizando funções `lambda`, `map`, `reduce` (do módulo `functools`) e funções puras, ou seja, funções que não alteram estado externo e sempre produzem o mesmo resultado para a mesma entrada.

Principais elementos funcionais utilizados:

- `media = lambda notas: ...`: função pura que calcula a média de uma lista de notas utilizando `reduce` para somar os valores;
- `situacao = lambda m: ...`: função pura que determina a situação (Aprovado/Reprovado) a partir da média, sem efeitos colaterais;
- `enriquecer = lambda aluno: {**aluno, ...}`: função pura que retorna um novo dicionário (imutabilidade) contendo os dados originais do aluno acrescidos de média e situação, sem alterar o dicionário recebido;
- `ordenar = lambda alunos: sorted(...)`: ordena a lista de alunos por média de forma decrescente, utilizando uma função-chave `lambda`;
- `maior_nota` e `menor_nota`: implementadas com `reduce`, percorrendo todas as notas da turma para encontrar os valores extremos sem uso de laços explícitos (`for`);
- `ler_aluno(i)`: utiliza `map(ler_nota, range(1, 4))` para aplicar a função `ler_nota` a cada uma das três posições de nota, em vez de um laço `for` tradicional;
- Pipeline funcional em `main()`: `alunos = ordenar(list(map(enriquecer, map(ler_aluno, range(1, n + 1)))))` — demonstra a composição de funções em cadeia (pipeline), em que a saída de uma função alimenta diretamente a entrada da próxima, sem variáveis intermediárias mutáveis.

É importante destacar que as funções de entrada (`ler_aluno`, `ler_nota`) e saída (`exibir_listagem`, `exibir_estatisticas`) concentram os efeitos colaterais do programa (leitura do teclado e escrita no console), enquanto o núcleo de processamento (média, situação, ordenação, agregações) é

composto inteiramente por funções puras — uma prática recomendada em programação funcional para isolar efeitos colaterais.

3.4 Principais Diferenças entre as Linguagens e Paradigmas

Do ponto de vista da implementação, observa-se que a versão Java exige maior volume de código e organização em múltiplos arquivos para representar o mesmo domínio que, em Python, é resolvido com estruturas de dados simples (dicionários e listas). Em contrapartida, a tipagem estática e a estrutura de classes de Java oferecem maior garantia de integridade dos dados em tempo de compilação, enquanto as versões em Python (estruturada e funcional) priorizam concisão e flexibilidade, ao custo de validações realizadas apenas em tempo de execução.

4. Análise Comparativa

4.1 Comparação Geral

Critério	Estruturado (Python)	OO (Java)	Funcional (Python)
Legibilidade	Alta — fluxo linear e direto	Média — exige navegar entre várias classes	Média — exige familiaridade com funções de alta ordem
Modularização	Boa — funções com responsabilidade única	Excelente — classes coesas e bem definidas	Boa — funções pequenas e independentes
Reutilização	Média — funções reutilizáveis, mas dados acoplados a dicionários	Alta — classes e interfaces reutilizáveis em outros contextos	Alta — funções puras facilmente reutilizadas/compostas
Facilidade de manutenção	Média — mudanças podem afetar várias funções	Alta — mudanças isoladas por classe (baixo acoplamento)	Alta — ausência de efeitos colaterais facilita prever impacto
Desempenho percebido	Rápido para o volume de dados do problema	Levemente mais lento na inicialização (JVM), mas robusto em escala	Equivalente ao estruturado; reduce/map têm overhead leve
Diferenças no comportamento das linguagens	Implementação rápida	Exige mais código boilerplate	xige raciocínio diferente, mas código compacto
Facilidade de implementação	Alta — natural para quem já programa	Média — exige planejamento prévio das classes	Média — exige adaptação ao raciocínio funcional

4.2 Tipos de Dados

No Python, a tipagem é dinâmica, ou seja, não precisa declarar o tipo das variáveis porque a linguagem identifica isso durante a execução. Já no Java, a tipagem é estática, então é obrigatório definir o tipo de cada variável, o que ajuda a evitar erros, mas deixa o código mais detalhado.

4.3 Escopo

Na versão estruturada, as variáveis ficam dentro das funções e os dados são passados por parâmetros. Na orientação a objetos, os dados pertencem aos objetos e são acessados por métodos, mantendo maior controle com o uso de `private`. Já na versão funcional, as variáveis ficam limitadas ao escopo das expressões lambda, evitando variáveis globais.

4.4 Uso e Armazenamento de Dados

Em todas as versões, os dados ficam armazenados apenas na memória durante a execução. No Python, foram usadas listas de dicionários e o acesso é feito diretamente pelas chaves. No Java, os dados ficam em objetos da classe `Aluno` dentro da classe `Turma`, sendo acessados por métodos, seguindo o conceito de encapsulamento.

4.5 Persistência de Dados

Nenhuma das versões salva os dados de forma permanente. Todas armazenam as informações apenas enquanto o programa está rodando, e tudo é perdido ao finalizar a execução. Futuramente, a versão em Java poderia adicionar persistência usando arquivos ou banco de dados.

4.6 Comportamento das Linguagens

O Python é interpretado, então o desenvolvimento é mais rápido porque não precisa compilar o código. Já o Java é compilado e possui tipagem forte, o que ajuda a encontrar erros antes da execução, mas exige mais organização e uma estrutura de código maior.

5. Discussão sobre Concorrência (Item Reflexivo)

Atualmente, as três versões funcionam de forma sequencial (uma tarefa por vez). Porém, elas podem ser adaptadas no futuro para suportar multitarefa, múltiplos usuários ou processamento concorrente, melhorando o desempenho da aplicação.

- Estruturado (Python): poderia ser adaptado utilizando o módulo multiprocessing para processar simultaneamente os dados de múltiplas turmas, já que as funções de cálculo (`calcular_media`, `calcular_estatisticas`) são, em sua maioria, isentas de efeitos colaterais sobre dados compartilhados;
- Orientado a Objetos (Java): a estrutura de classes facilita a evolução para um cenário concorrente, por exemplo, utilizando threads para processar o cadastro de múltiplas turmas em paralelo, com sincronização (`synchronized`) nos métodos que alteram o estado compartilhado de Turma;
- Funcional (Python): é o paradigma mais naturalmente propício à concorrência, pois as funções puras (`media`, `situacao`, `enriquecer`) não possuem efeitos colaterais nem dependem de estado compartilhado mutável, permitindo paralelizar o processamento de múltiplos alunos com segurança (por exemplo, usando `concurrent.futures.map` em vez de `map` nativo).

6. Conclusão

6.1 Paradigma Mais Adequado

Para esse sistema de notas e médias, a programação orientada a objetos foi a mais adequada, pois facilita a organização do código, representa melhor o problema e torna a manutenção e futuras melhorias mais simples.

6.2 Vantagens Observadas

- **Estruturado:** simples, fácil de aprender e rápido de implementar.
- **Orientado a Objetos:** melhor organização, reutilização de código e manutenção mais fácil.
- **Funcional:** código mais enxuto, previsível e sem efeitos colaterais nas funções.

6.3 Dificuldades Encontradas

- Na versão funcional, a maior dificuldade foi substituir os laços for por map, filter e reduce.
- No Java, foi definir uma estrutura de classes sem deixar o projeto complexo.
- Também foi necessário manter as mesmas regras de negócio em todas as versões para garantir uma comparação justa.

6.4 Limitações da Solução

- O sistema não salva os dados após ser fechado, possui apenas as validações necessárias e funciona somente pelo terminal, sem interface gráfica.

6.5 Principais Aprendizados da Disciplina

O trabalho ajudou a entender melhor as diferenças entre os paradigmas estruturado, orientado a objetos e funcional, mostrando como cada um influencia a organização do código, o tratamento dos dados e a manutenção do sistema. Também reforçou a importância de conceitos como tipagem, escopo e imutabilidade no desenvolvimento de software.

Anexo — Códigos-Fonte

Esta seção apresenta o código-fonte completo das três implementações desenvolvidas para o Problema 1 — Sistema de Notas e Médias de Alunos.

A.1 Implementação Estruturada (Python)

Trabalho_de_paradigmas_-_Sistemas_de_notas_(Estruturado).py

```
# =====
# SISTEMAS DE NOTAS - PARADIGMA ESTRUTURADO
# Uso de funções, estruturas de controle e modularização
# =====

def ler_alunos(n):
    """Lê nome e notas de n alunos e retorna uma lista de dicionário"""
    alunos = []
    for i in range(1, n + 1):
        print(f"\n--- Aluno {i} ---")
        nome = input("Nome: ").strip()
        notas = []
        for j in range(1, 4):
            while True:
                try:
                    nota = float(input(f"Nota {j} (0-10): "))
                    if 0 <= nota <= 10:
                        notas.append(nota)
                        break
                    print(" Nota deve estar entre 0 e 10.")
                except ValueError:
                    print(" Digite um número válido.")
            alunos.append({"nome": nome, "notas": notas})
    return alunos

def calcular_media(notas):
    """Retorna a média aritmética de uma lista de notas."""
    return sum(notas) / len(notas)

def determinar_situacao(media):
    """Retorna 'Aprovado' se média >= 6, caso contrário 'Reprovado'. """
    return "Aprovado" if media >= 6 else "Reprovado"

def processar_alunos(alunos):
    """Adicionar média e situação a cada aluno."""
    for aluno in alunos:
        aluno["média"] = calcular_media(aluno["notas"])
        aluno["situacao"] = determinar_situacao(aluno["média"])
    return alunos

def ordenar_alunos(alunos):
    """Retorna lista ordenada por média (descrescente). """
    return sorted(alunos, key=lambda a: a["média"], reverse=True)

def exibir_listagem(alunos):
    """Exibe a tabela de resultados individuais."""
    print("\n" + "=" * 60)
    print(f"{'ALUNOS': <25} {'MÉDIA': >6} {'SITUAÇÃO'}")
    print("-" * 60)
    for aluno in alunos:
        print(f"{aluno['nome']:<25} {aluno['média']:>6.2f} {aluno['situacao']}")
    print("=" * 60)
```

```

def calcular_estatisticas(alunos):
    """Calcular e exibe média geral, maior e menor nota da turma."""
    todas_as_notas = [n for a in alunos for n in a["notas"]]
    media_geral = sum(a["média"] for a in alunos) / len(alunos)
    maior_nota = max(todas_as_notas)
    menor_nota = min(todas_as_notas)

    print("\n ESTATISTICAS DA TURMA")
    print(f" Média geral : {media_geral:.2f}")
    print(f" Maior nota : {maior_nota:.2f}")
    print(f" Menor nota : {menor_nota:.2f}")
    print("=" * 60)

def main():
    print("=" * 40)
    print("SITESMAS DE NOTAS - ALUNOS")
    print("=" * 40)

    while True:
        try:
            n = int(input("\nQuantos alunos deseja cadastrar? "))
            if n > 0:
                break
            print(" DIGite um número maior que zero.")
        except ValueError:
            print("Valor inválido.")

    alunos = ler_alunos(n)
    alunos = processar_alunos(alunos)
    ordenados = ordenar_alunos(alunos)

    exibir_listagem(ordenados)
    calcular_estatisticas(alunos)

if __name__ == "__main__":
    main()

```

A.2 Implementação Declarativa/Funcional (Python)

Trabalho_de_paradigmas_-_Sistemas_de_notas_(Declarativo).py

```
# =====
# SISTEMA DE NOTAS – PARADIGMA DECLARATIVO / FUNCIONAL
# Uso de funções puras, map, filter, lambda, reduce
# =====

from functools import reduce

# - Função puras (sem efeitos colaterais) -----

media      = lambda notas: reduce(lambda acc, n: acc + n, notas) / len(notas)
situacao   = lambda m: "Aprovado" if m >= 6 else "Reprovado"
enriquecer = lambda aluno: {**aluno,
                             "media": media(aluno["notas"]),
                             "situacao": situacao(media(aluno["notas"]))}
ordenar     = lambda alunos: sorted(alunos, key=lambda a: a["media"], reverse=True)

todas_notas = lambda alunos: [n for a in alunos for n in a["notas"]]
media_geral = lambda alunos: media(list(map(lambda a: a["media"], alunos)))
maior_nota  = lambda alunos: reduce(lambda acc, n: n if n > acc else acc, todas_notas(alunos))
menor_nota  = lambda alunos: reduce(lambda acc, n: n if n < acc else acc, todas_notas(alunos))

# ----- Entrada de dados -----

def ler_nota(j):
    while True:
        try:
            n = float(input(f" Nota {j} (0-10): "))
            if 0 <= n <= 10:
                return n
            print(" Nota deve estar entre 0 a 10.")
        except ValueError:
            print(" Digite um número válido.")

def ler_aluno(i):
    print(f"\n--- Aluno {i} ---")
    return {
        "nome": input("Nome: ").strip(),
        "notas": list(map(ler_nota, range(1, 4))) # map aplica ler_nota para [1, 2, 3]
    }

def ler_quantidade():
    while True:
        try:
            n = int(input("\nQuantos alunos deseja cadastrar? "))
            if n > 0:
                return n
            print("Digite um número maior que zero.")
        except ValueError:
            print("Valor inválido.")

# --- Saída de dados (único lugar com efeitos colaterais) -----

def exibir_listagem(alunos):
    print("\n" + "=" * 60)
    print(f"{'ALUNO':<25} {'MÉDIA':>6} {'SITUAÇÃO'}")
    print("=" * 60)
    list(map(
        lambda a: print(f"{a['nome']:<25} {a['media']:>6.2f} {a['situacao']}"), alunos
```

```

))
print("=" * 60)

def exibir_estatisticas(alunos):
    print("\n ESTATÍSTICAS DA TURMA")
    print(f" Média geral : {media_geral(alunos):.2f}")
    print(f" Maior nota : {maior_nota(alunos):.2f}")
    print(f" Menor nota : {menor_nota(alunos):.2f}")
    print("=" * 40)

# ----- Pipeline principal -----

def main():
    print("="*40)
    print("SISTEMAS DE NOTAS - ALUNOS")
    print("="*40)

    n = ler_quantidade()

    # Pipeline declarativo: leitura -> enriquecimento -> ordenação
    alunos = ordenar(
        list(map(enriquecer,
                 map(ler_aluno, range(1, n + 1))))
    )

    exibir_listagem(alunos)
    exibir_estatisticas(alunos)

if __name__ == "__main__":
    main()

```

A.3 Implementação Orientada a Objetos (Java)

Pessoa.java

```
/**
 * Classe abstrata base que representa uma Pessoa.
 * Demonstra: abstração, encapsulamento e herança.
 */
public abstract class Pessoa {

    private String nome;

    public Pessoa(String nome) {
        if (nome == null || nome.trim().isEmpty()) {
            throw new IllegalArgumentException("Nome não pode ser vazio ou nulo.");
        }
        this.nome = nome.trim();
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        if (nome == null || nome.trim().isEmpty()) {
            throw new IllegalArgumentException("Nome não pode ser vazio ou nulo.");
        }
        this.nome = nome.trim();
    }

    /**
     * Método abstrato – cada subclasse deve fornecer
     * sua própria representação em String.
     * Demonstra: polimorfismo (método abstrato).
     */
    public abstract String exibirInformacoes();

    @Override
    public String toString() {
        return exibirInformacoes();
    }
}
```

Aluno.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Representa um Aluno com suas notas.
 * Demonstra: herança (extends Pessoa), encapsulamento e polimorfismo.
 */
public class Aluno extends Pessoa implements Comparable<Aluno> {

    private static final double MEDIA_APROVACAO = 6.0;

    private List<Double> notas;

    public Aluno(String nome) {
        super(nome);
        this.notas = new ArrayList<>();
    }

    /**
     * Adiciona uma nota ao aluno com validação.
     * Demonstra: tratamento de exceções (lança NotaInvalidaException).
     */
}
```



```

public void adicionarNota(double nota) throws NotaInvalidaException {
    if (nota < 0.0 || nota > 10.0) {
        throw new NotaInvalidaException(nota);
    }
    notas.add(nota);
}

public List<Double> getNotas() {
    return Collections.unmodifiableList(notas);
}

/**
 * Calcula a média aritmética das notas.
 */
public double calcularMedia() {
    if (notas.isEmpty()) {
        return 0.0;
    }
    double soma = 0.0;
    for (double nota : notas) {
        soma += nota;
    }
    return soma / notas.size();
}

/**
 * Retorna a maior nota do aluno.
 */
public double getMaiorNota() {
    if (notas.isEmpty()) return 0.0;
    return Collections.max(notas);
}

/**
 * Retorna a menor nota do aluno.
 */
public double getMenorNota() {
    if (notas.isEmpty()) return 0.0;
    return Collections.min(notas);
}

/**
 * Determina a situação do aluno.
 * Demonstra: polimorfismo (comportamento específico de Aluno).
 */
public String getSituacao() {
    return calcularMedia() >= MEDIA_APROVACAO ? "Aprovado ✓" : "Reprovado X";
}

public boolean isAprovado() {
    return calcularMedia() >= MEDIA_APROVACAO;
}

/**
 * Ordena alunos por nome (para Collections.sort).
 * Demonstra: implementação de interface Comparable.
 */
@Override
public int compareTo(Aluno outro) {
    return this.getNome().compareToIgnoreCase(outro.getNome());
}

/**
 * Implementação do método abstrato de Pessoa.
 * Demonstra: polimorfismo (override de método abstrato).
 */
@Override
public String exibirInformacoes() {
    StringBuilder sb = new StringBuilder();
    sb.append(String.format("%-25s", getNome()));
    sb.append(String.format(" | Média: %5.2f", calcularMedia()));
}

```

```

        sb.append(String.format(" | Situação: %-12s", getSituacao()));

        sb.append(" | Notas: [");
        for (int i = 0; i < notas.size(); i++) {
            sb.append(String.format("%.1f", notas.get(i)));
            if (i < notas.size() - 1) sb.append(", ");
        }
        sb.append("]");

        return sb.toString();
    }
}

```

NotaInvalidaException.java

```

/**
 * Exceção personalizada para notas inválidas.
 * Demonstra: tratamento de exceções com classe própria.
 */
public class NotaInvalidaException extends Exception {

    private double notaInformada;

    public NotaInvalidaException(double notaInformada) {
        super(String.format(
            "Nota inválida: %.2f. As notas devem estar entre 0.0 e 10.0.",
            notaInformada
        ));
        this.notaInformada = notaInformada;
    }

    public double getNotaInformada() {
        return notaInformada;
    }
}

```

Turma.java

```

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Representa uma Turma com uma coleção de Alunos.
 * Encapsula a lógica de estatísticas da turma.
 * Demonstra: encapsulamento e composição.
 */
public class Turma {

    private String nomeTurma;
    private List<Aluno> alunos;

    public Turma(String nomeTurma) {
        if (nomeTurma == null || nomeTurma.trim().isEmpty()) {
            throw new IllegalArgumentException("Nome da turma não pode ser vazio.");
        }
        this.nomeTurma = nomeTurma.trim();
        this.alunos = new ArrayList<>();
    }

    public void adicionarAluno(Aluno aluno) {
        if (aluno == null) {
            throw new IllegalArgumentException("Aluno não pode ser nulo.");
        }
        alunos.add(aluno);
    }
}

```

```

public String getNomeTurma() {
    return nomeTurma;
}

public int getTotalAlunos() {
    return alunos.size();
}

/**
 * Retorna lista de alunos ordenada por nome.
 */
public List<Aluno> getAlunosOrdenados() {
    List<Aluno> ordenados = new ArrayList<>(alunos);
    Collections.sort(ordenados);
    return ordenados;
}

/**
 * Calcula a média geral de todos os alunos da turma.
 */
public double calcularMediaGeral() {
    if (alunos.isEmpty()) return 0.0;
    double soma = 0.0;
    for (Aluno a : alunos) {
        soma += a.calcularMedia();
    }
    return soma / alunos.size();
}

/**
 * Retorna a maior nota registrada na turma.
 */
public double getMaiorNotaTurma() {
    if (alunos.isEmpty()) return 0.0;
    double maior = Double.MIN_VALUE;
    for (Aluno a : alunos) {
        if (!a.getNotas().isEmpty() && a.getMaiorNota() > maior) {
            maior = a.getMaiorNota();
        }
    }
    return maior == Double.MIN_VALUE ? 0.0 : maior;
}

/**
 * Retorna a menor nota registrada na turma.
 */
public double getMenorNotaTurma() {
    if (alunos.isEmpty()) return 0.0;
    double menor = Double.MAX_VALUE;
    for (Aluno a : alunos) {
        if (!a.getNotas().isEmpty() && a.getMenorNota() < menor) {
            menor = a.getMenorNota();
        }
    }
    return menor == Double.MAX_VALUE ? 0.0 : menor;
}

/**
 * Conta quantos alunos foram aprovados.
 */
public long getTotalAprovados() {
    long count = 0;
    for (Aluno a : alunos) {
        if (a.isAprovado()) count++;
    }
    return count;
}

/**
 * Conta quantos alunos foram reprovados.
 */
public long getTotalReprovados() {

```

```

        return alunos.size() - getTotalAprovados();
    }
}

```

Relatorio.java

```

/**
 * Classe abstrata que define o contrato para geração de relatórios.
 * Demonstra: abstração e polimorfismo via método template.
 */
public abstract class Relatorio {

    protected Turma turma;

    public Relatorio(Turma turma) {
        if (turma == null) {
            throw new IllegalArgumentException("Turma não pode ser nula.");
        }
        this.turma = turma;
    }

    /**
     * Gera e exibe o relatório.
     * Padrão Template Method: define o esqueleto e delega partes às subclasses.
     */
    public final void gerar() {
        exibirCabecalho();
        exibirCorpo();
        exibirRodape();
    }

    protected abstract void exibirCabecalho();
    protected abstract void exibirCorpo();
    protected abstract void exibirRodape();

    // Utilitário compartilhado
    protected void linha(char caractere, int tamanho) {
        System.out.println(String.valueOf(caractere).repeat(tamanho));
    }
}

```

RelatorioTurma.java

```

import java.util.List;

/**
 * Relatório completo da turma.
 * Demonstra: herança e polimorfismo (estende Relatorio e implementa métodos abstratos).
 */
public class RelatorioTurma extends Relatorio {

    public RelatorioTurma(Turma turma) {
        super(turma);
    }

    @Override
    protected void exibirCabecalho() {
        linha('=', 80);
        System.out.printf("%35s\n", "SISTEMA DE NOTAS E MÉDIAS");
        System.out.printf("  Turma : %s\n", turma.getNomeTurma());
        System.out.printf("  Total de alunos: %d\n", turma.getTotalAlunos());
        linha('=', 80);
    }

    @Override
    protected void exibirCorpo() {
        System.out.println("\n  LISTAGEM DE ALUNOS (ordenada por nome)\n");
    }
}

```

```

        linha('-', 80);

        List<Aluno> ordenados = turma.getAlunosOrdenados();

        if (ordenados.isEmpty()) {
            System.out.println(" Nenhum aluno cadastrado.");
        } else {
            for (int i = 0; i < ordenados.size(); i++) {
                // Polimorfismo: chama exibirInformacoes() de Aluno via toString() de Pessoa
                System.out.printf(" %2d. %s%n", i + 1, ordenados.get(i));
            }
        }

        linha('-', 80);
    }

    @Override
    protected void exibirRodape() {
        System.out.println("\n ESTATÍSTICAS DA TURMA\n");
        linha('-', 80);
        System.out.printf(" %-30s %.2f%n", "Média Geral da Turma:", turma.calcularMediaGeral());
        System.out.printf(" %-30s %.2f%n", "Maior Nota:", turma.getMaiorNotaTurma());
        System.out.printf(" %-30s %.2f%n", "Menor Nota:", turma.getMenorNotaTurma());
        System.out.printf(" %-30s %d aluno(s)%n", "Total Aprovados:", turma.getTotalAprovados());
        System.out.printf(" %-30s %d aluno(s)%n", "Total Reprovados:", turma.getTotalReprovados());
        linha('=', 80);
        System.out.println();
    }
}

```

Main.java

```

import java.util.InputMismatchException;
import java.util.Scanner;

/**
 * Classe principal – ponto de entrada do sistema.
 * Demonstra: tratamento de exceções (try-catch-finally, exceções customizadas).
 */
public class Main {

    private static Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        System.out.println("=====");
        System.out.println(" SISTEMA DE NOTAS E MÉDIAS DE ALUNOS");
        System.out.println("=====\\n");

        Turma turma = null;

        try {
            // Lê nome da turma
            System.out.print("Nome da turma: ");
            String nomeTurma = scanner.nextLine();
            turma = new Turma(nomeTurma);

            // Lê quantidade de alunos
            int n = lerInteiroPositivo("Número de alunos: ");

            // Lê quantidade de notas por aluno
            int qtdNotas = lerInteiroPositivo("Quantidade de notas por aluno: ");

            System.out.println();

            // Cadastra cada aluno
            for (int i = 1; i <= n; i++) {
                System.out.println("—— Aluno " + i + " —————");
                Aluno aluno = cadastrarAluno(qtdNotas);
            }
        } catch (InputMismatchException e) {
            System.out.println("Erro: Entrada inválida. Por favor, digite um número inteiro positivo.");
        }
    }

    private static int lerInteiroPositivo(String mensagem) {
        int valor;
        do {
            System.out.print(mensagem);
            valor = scanner.nextInt();
            if (valor < 1) {
                System.out.println("Erro: Valor inválido. Digite um número inteiro positivo.");
            }
        } while (valor < 1);
        return valor;
    }
}

```

```

        turma.adicionarAluno(aluno);
        System.out.println("  ✓ Aluno cadastrado com sucesso!\n");
    }

    // Gera relatório - polimorfismo: RelatorioTurma é tratado como Relatorio
    Relatorio relatorio = new RelatorioTurma(turma);
    relatorio.gerar();

} catch (IllegalArgumentException e) {
    System.err.println("\n[ERRO DE CONFIGURAÇÃO] " + e.getMessage());
} finally {
    scanner.close();
    System.out.println("Sistema encerrado.");
}
}

/**
 * Cadastra um aluno lendo nome e notas do console.
 * Demonstra: tratamento de NotaInvalidaException com reentrada.
 */
private static Aluno cadastrarAluno(int qtdNotas) {
    Aluno aluno = null;

    // Lê nome com validação
    while (aluno == null) {
        try {
            System.out.print(" Nome do aluno: ");
            String nome = scanner.nextLine();
            aluno = new Aluno(nome);
        } catch (IllegalArgumentException e) {
            System.err.println(" [ERRO] " + e.getMessage() + " Tente novamente.");
        }
    }

    // Lê notas com validação
    for (int j = 1; j <= qtdNotas; j++) {
        boolean notaValida = false;
        while (!notaValida) {
            try {
                System.out.printf(" Nota %d (0.0 a 10.0): ", j);
                double nota = scanner.nextDouble();
                aluno.adicionarNota(nota); // lança NotaInvalidaException se inválida
                notaValida = true;
            } catch (NotaInvalidaException e) {
                // Exceção de domínio específica
                System.err.println(" [NOTA INVÁLIDA] " + e.getMessage());
            } catch (InputMismatchException e) {
                // Entrada não numérica
                System.err.println(" [ERRO] Digite um número válido (ex: 7.5).");
                scanner.nextLine(); // limpa buffer
            }
        }
        scanner.nextLine(); // consome o '\n' após nextDouble()
    }

    return aluno;
}

/**
 * Lê um inteiro positivo com tratamento de exceção.
 */
private static int lerInteiroPositivo(String mensagem) {
    int valor = 0;
    boolean valido = false;
    while (!valido) {
        try {
            System.out.print(mensagem);
            valor = scanner.nextInt();
            scanner.nextLine();
            if (valor <= 0) {
                System.err.println(" [ERRO] O valor deve ser maior que zero.");
            }
        }
    }
}

```

```
        } else {
            valido = true;
        }
    } catch (InputMismatchException e) {
        System.err.println(" [ERRO] Digite um número inteiro válido.");
        scanner.nextLine(); // limpa buffer
    }
}
return valor;
}
```